Jackfruit Phase-1

Software Requirements Specification for Hostel Management System

Version 2.0 (Final) Prepared by: Member 1 Tejasvi K S – PES1UG24CS499 / Member 2 Prathiksha B P – PES1UG24CS925 / Member 3 Vilohith – PES1UG24CS535 / Member 4 Varsha – PES1UG24CS516

Group Number: G-03 | **Project ID:** HMS-2026

Course: UE24CS341A – Software Engineering, V Semester

Organization: PES University, Bangalore — Dept. of CSE

Date Created: 16/9/26 | **Last Updated:** 8/11/26

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. Analysis Models
- 5. System Features

- 5.1 Student & Access Management

- 5.2 Room & Allocation Management
- 5.3 Fee Management
- 5.4 Complaints & Leave Management
- 6. Other Nonfunctional Requirements
- 7. Other Requirements
- Appendix A: Glossary
- Appendix B: Field Layouts
- Appendix C: Requirement Traceability Matrix
- Appendix D: Use Cases
- Appendix E: Changes After the Phase-1 Baseline

Revision history

Name	Date	Reason For Changes	Version
Tejasvi K S	16/9/26	Initial SRS scope and document	0.1
Vilohith, Varsha, Prathiksha	3/10/26	Updated their respective sections in SRS	1.0
Vilohith, Varsha, Prathiksha	5/10/26	Made required changes to SRS	1.1
Tejasvi K S	11/10/26	Fixed minor wordings and table structure	1.2
Tejasvi K S, Prathiksha B P, Vilohith, Varsha	12/10/26	Added REQ-21, NFR-2 to NFR-5, security objectives, SEC-5, SEC-6, second use case diagram and traceability matrix	1.3

Name	Date	Reason For Changes	Version
Tejasvi K S, Prathiksha B P, Vilohith, Varsha	8/11/26	Final version after Sprint 2: aligned with the implemented system, traceability matrix updated with executed test cases, data layouts updated (Allocation, AuditEntry), changes after the baseline recorded in Appendix E	2.0

1. Introduction

1.1 Purpose

The Software Requirements Specification (SRS) defines the functional and nonfunctional requirements for Version 2.0 of the Hostel Management System. The system is a web-based application developed for the Software Engineering course at PES University to manage student information, hostel room allocation, fee payment records, complaints, and leave requests for a single hostel. The system provides separate functionality for Student and Admin users.

1.2 Intended Audience and Reading Suggestions

This document is intended for the Hostel Management System development team, the Software Engineering course teacher/evaluator, and testers responsible for verifying the system. The SRS is organized into the Introduction, Overall Description, External Interface Requirements, Analysis Models, System Features, Other Nonfunctional Requirements, Other Requirements, and Appendices.

1.3 Product Scope

The Hostel Management System is intended to replace manual register- and notebook-based management of student information, hostel room allocation, fee records, complaints, and leave requests with a centralized web application. The system is intended for students and administrators managing a single hostel. It supports two user roles only: Student and Admin. The Student can manage their profile, view room and fee information, submit complaints, and submit leave requests, while the Admin can manage student accounts, rooms, room allocation, fee records, complaints, and leave requests. The system does not include a Warden role, visitor management, a room-change request workflow, online payment gateway integration, receipt/PDF generation, or multi-hostel management.

1.4 References

The following references were used in preparing this Software Requirements Specification:

1. Jackfruit — Software Engineering Project Guidelines 2026.
2. Django Documentation — documentation for the Django web framework used by the project.
3. PostgreSQL Documentation — documentation for the PostgreSQL database management system used by the project.

2. Overall Description

2.1 Product Perspective

The Hostel Management System is a new, self-contained, standalone web application developed specifically for this course project. It is not a follow-on to, or replacement of, any existing production system, and it does not integrate with any larger institutional system. It is being built as a mini-project for the Software Engineering course (UE24CS341A) and is scoped as a demonstration system, not a production deployment.

2.2 Product Functions

- Student registration, login, and profile management
- Room and block setup, and room allocation/deallocation
- Fee structure setup and payment record tracking
- Complaint filing/tracking and leave request/approval

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Student	Register, view/edit own profile, view room, view fee dues, file complaints, submit leave requests	Non-technical, occasional use
Admin	Verify student accounts, manage rooms, allocate/deallocate rooms, define fee structure, record payments, manage complaints, approve/reject leave	Higher privilege, trained user

(Only two user classes.)

2.4 Operating Environment

The system is a web application accessed through any modern browser (Chrome, Firefox, Edge). It is built using Python 3.12 with the Django 5.2 web framework, and uses PostgreSQL 16 as the backend database. It runs locally with Docker Compose (application and PostgreSQL containers). For quick local development without Docker, the application can also run on a local SQLite database; PostgreSQL is used in Docker and in CI. No client-side installation is required beyond a browser.

2.5 Design and Implementation Constraints

- Must be built using Python (Django), per the team's chosen tech stack.
- Single hostel/single campus scope — no multi-hostel support.
- Two user roles only: Student and Admin. No payment gateway integration — fee payments are admin-recorded entries, not processed transactions.
- Agile/Scrum methodology mandatory; GitHub used for backlog, issues, and pull requests, with CI (GitHub Actions) running tests on every PR.

2.6 Assumptions and Dependencies

- Assumes an admin verifies new student accounts before a room can be allocated to them.

- Assumes fee payments happen outside the system (cash/bank transfer) and are simply recorded by the admin — the system does not process, authorize, or validate any actual payment transaction.
- Depends on Django's built-in authentication and ORM; no other major third-party service is required for this scope.

---

3. External Interface Requirements

3.1 User Interfaces

The system is accessed through a browser-based interface. All screens share the same layout: a top bar with the system name and the logged-in user, a left navigation menu that shows only the pages permitted for the user's role, and a main content area. The figures in this section are UI prototypes of the planned screens; the implemented screens may differ in minor styling.

Formatting conventions used consistently across all screens:

- Room status is shown as a coloured badge: green "Vacant" and red "Occupied". A room whose occupancy equals its capacity is shown as "Occupied"; otherwise it is "Vacant".
- Occupancy is displayed as "current / capacity" (for example, "1 / 2").
- Complaint and leave statuses are also shown as badges: Open (red), In-progress (blue), Resolved (green); Pending (amber), Approved (green), Rejected (red).
- Form validation errors are displayed in red text directly below the field that caused them, and the field is outlined in red.
- Success and failure messages for actions are displayed in a green or red banner at the top of the page.
- Admin-only screens and actions are not shown to Student users.

3.1.1 Student Dashboard

The landing page for a logged-in student. It summarises the student's allocated room, outstanding fee, open complaints, and leave requests, and shows the student's profile.

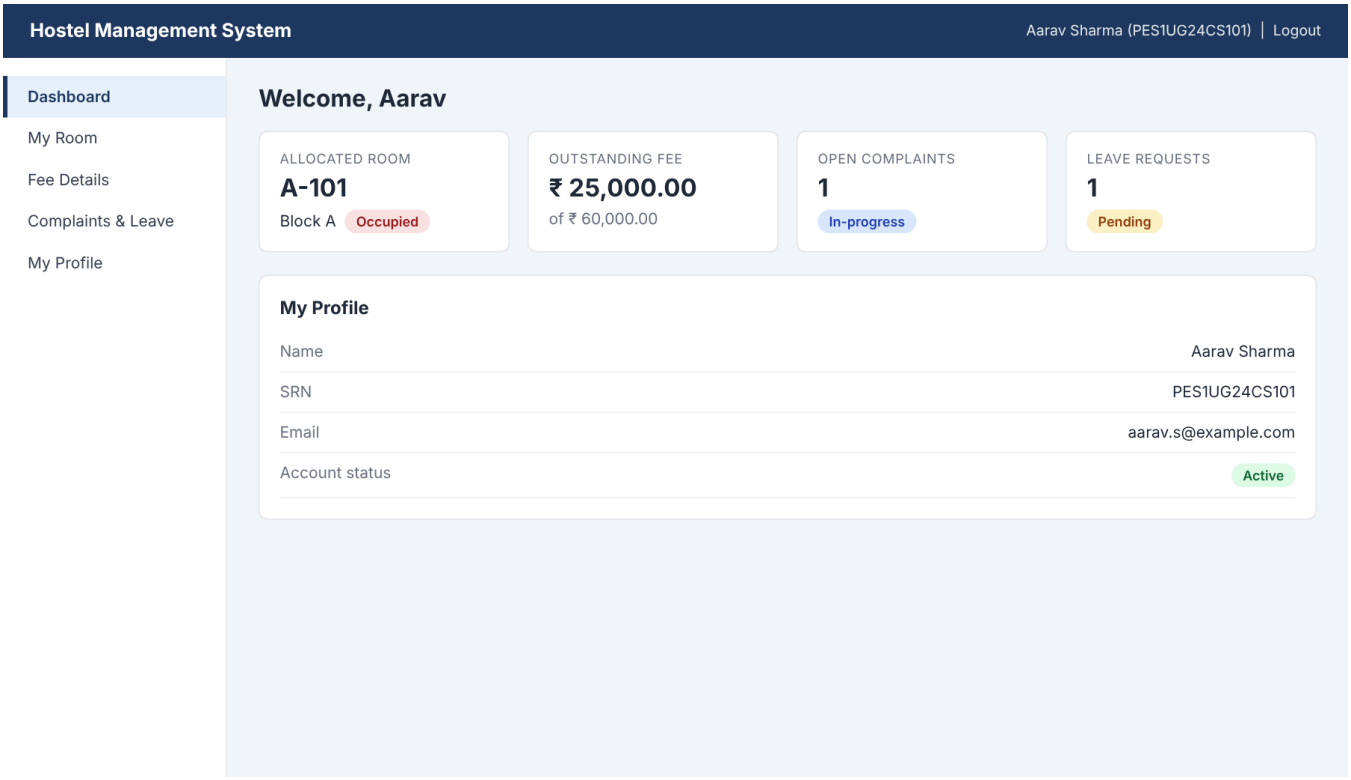


Figure 3.1 – Student Dashboard.

3.1.2 My Room (Student)

A read-only view of the student's allocated block, room number, room type, capacity, occupancy, and status. If no room is allocated, the screen displays "No room allocated yet."

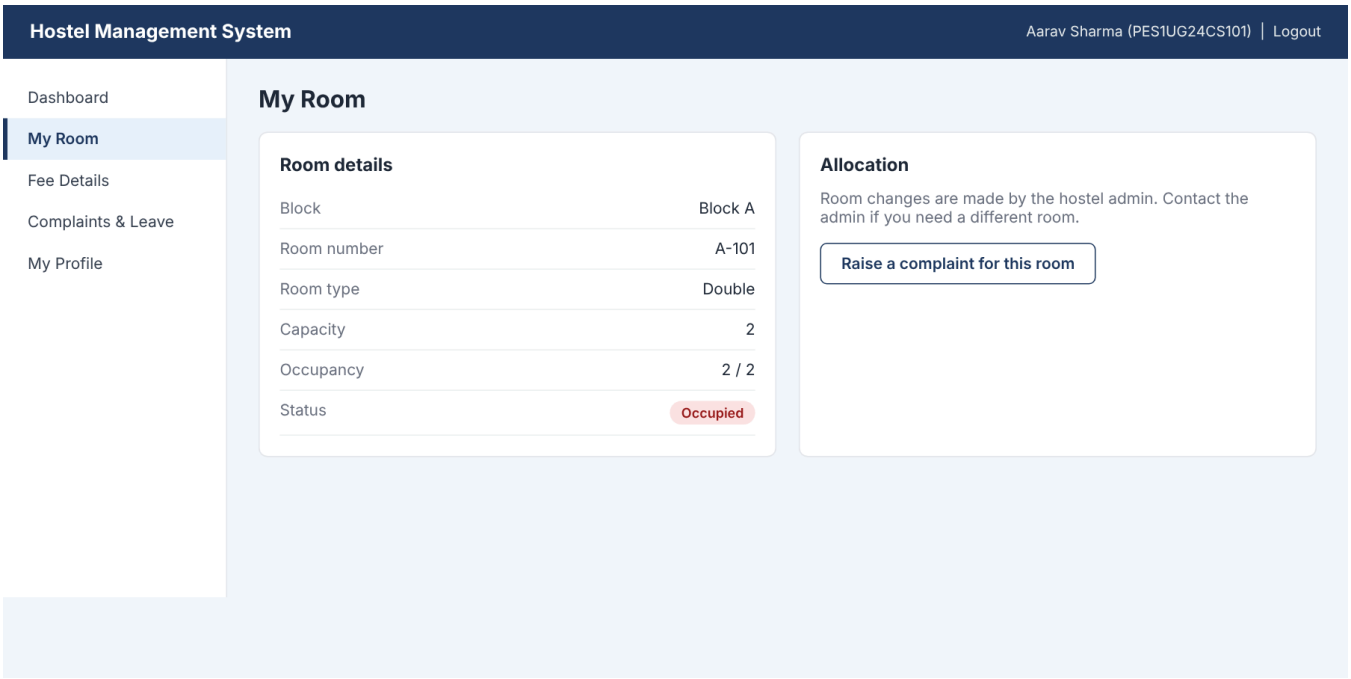


Figure 3.2 – My Room screen.

3.1.3 Admin Dashboard

The landing page for the admin. It shows summary counts and the list of pending student registrations, which the admin can approve or reject.

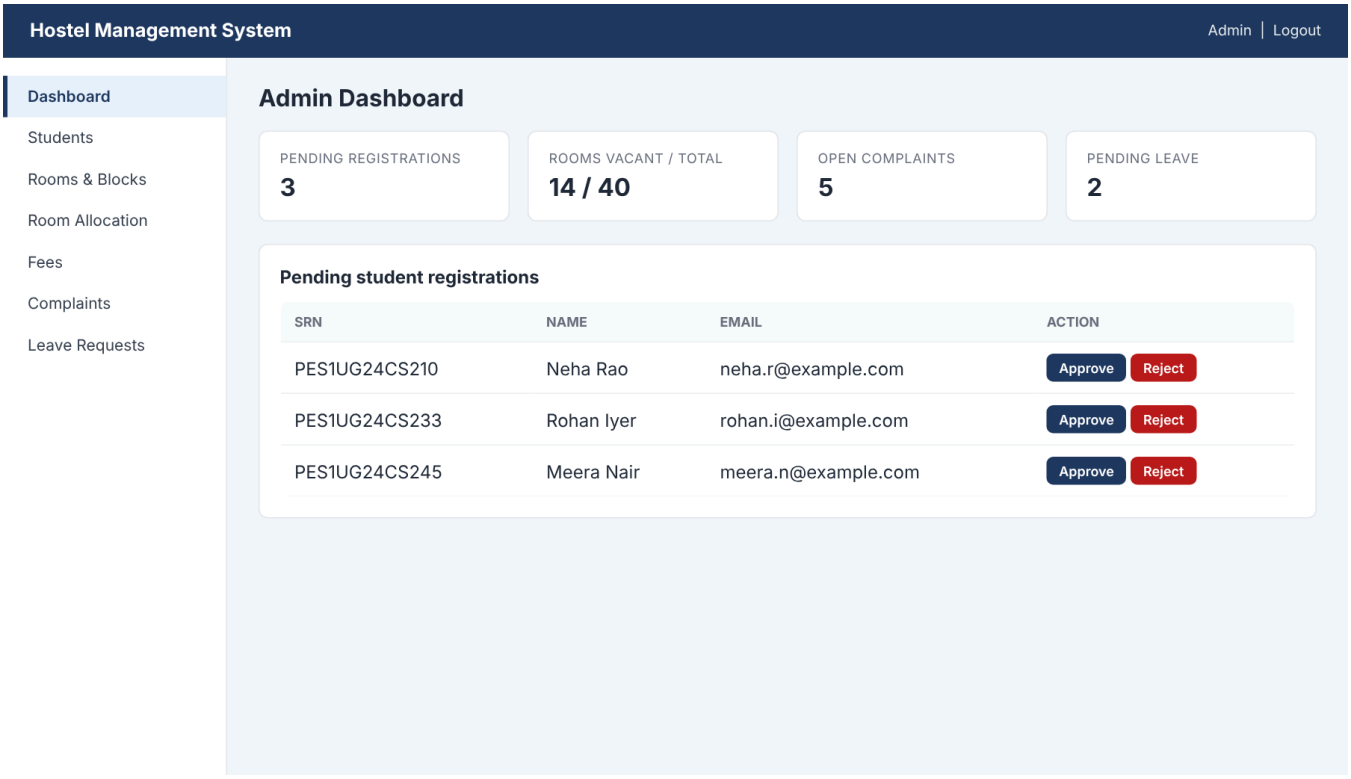


Figure 3.3 – Admin Dashboard.

3.1.4 Rooms & Blocks (Admin)

Lists all rooms with block, type, capacity, occupancy, and status, and provides a form to add or edit a room or block. The figure shows a success banner and a validation error on the Capacity field.

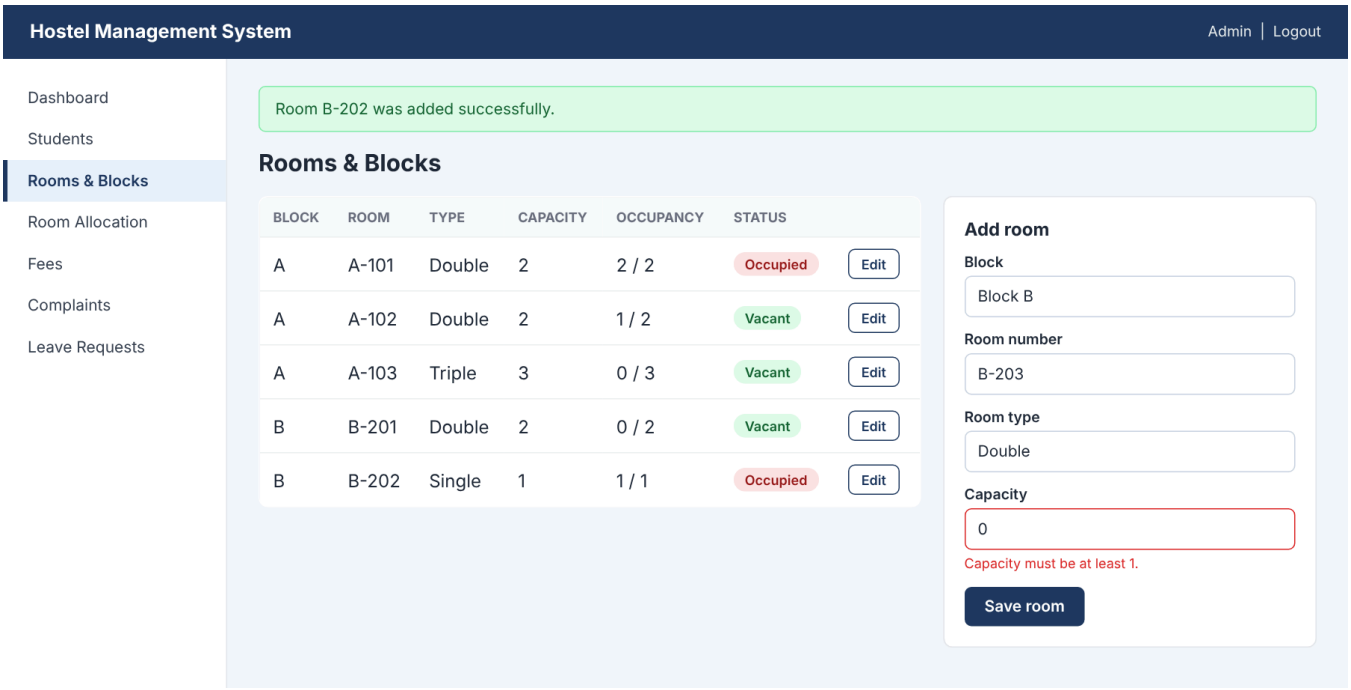


Figure 3.4 – Rooms & Blocks screen.

3.1.5 Room Allocation (Admin)

Allows the admin to select a verified student and a room and to allocate or deallocate. An attempt to allocate a room that is already at capacity is rejected with an error message and no allocation is made.

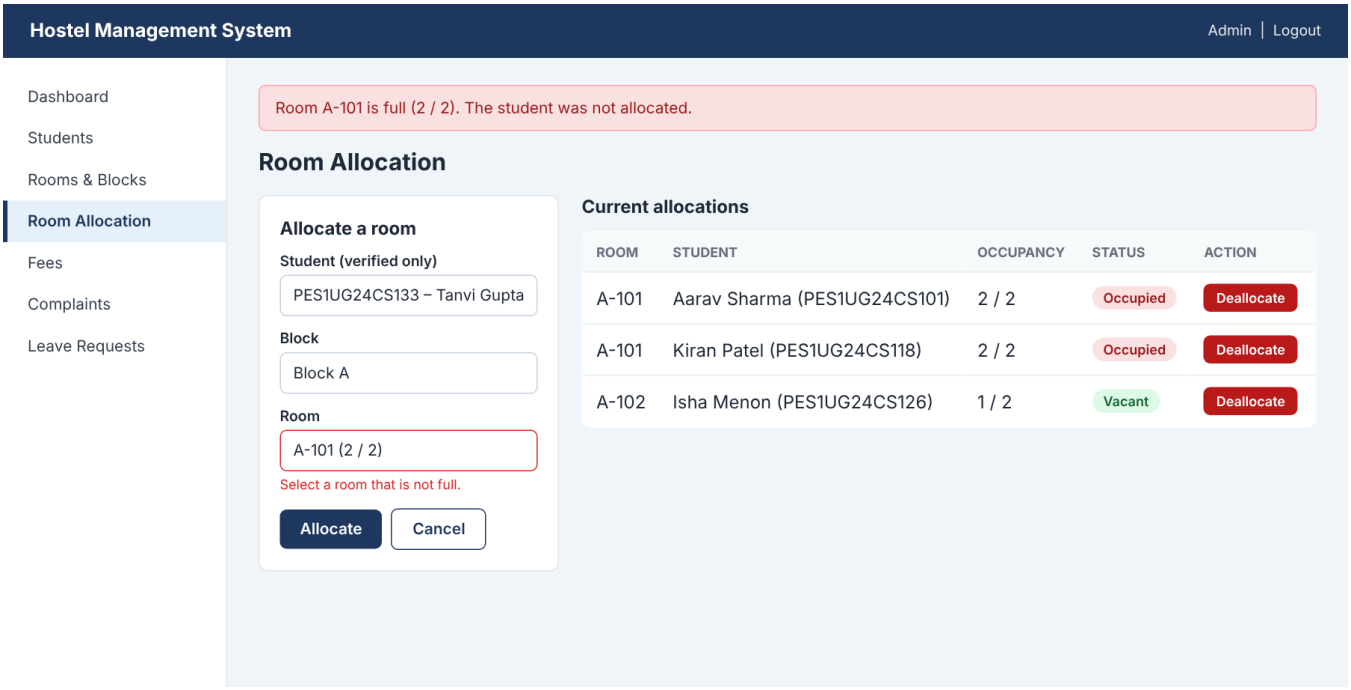


Figure 3.5 – Room Allocation screen showing a "room is full" error.

3.1.6 Fee Details (Student)

Shows the student's fee amount, total paid, outstanding balance, and payment history.

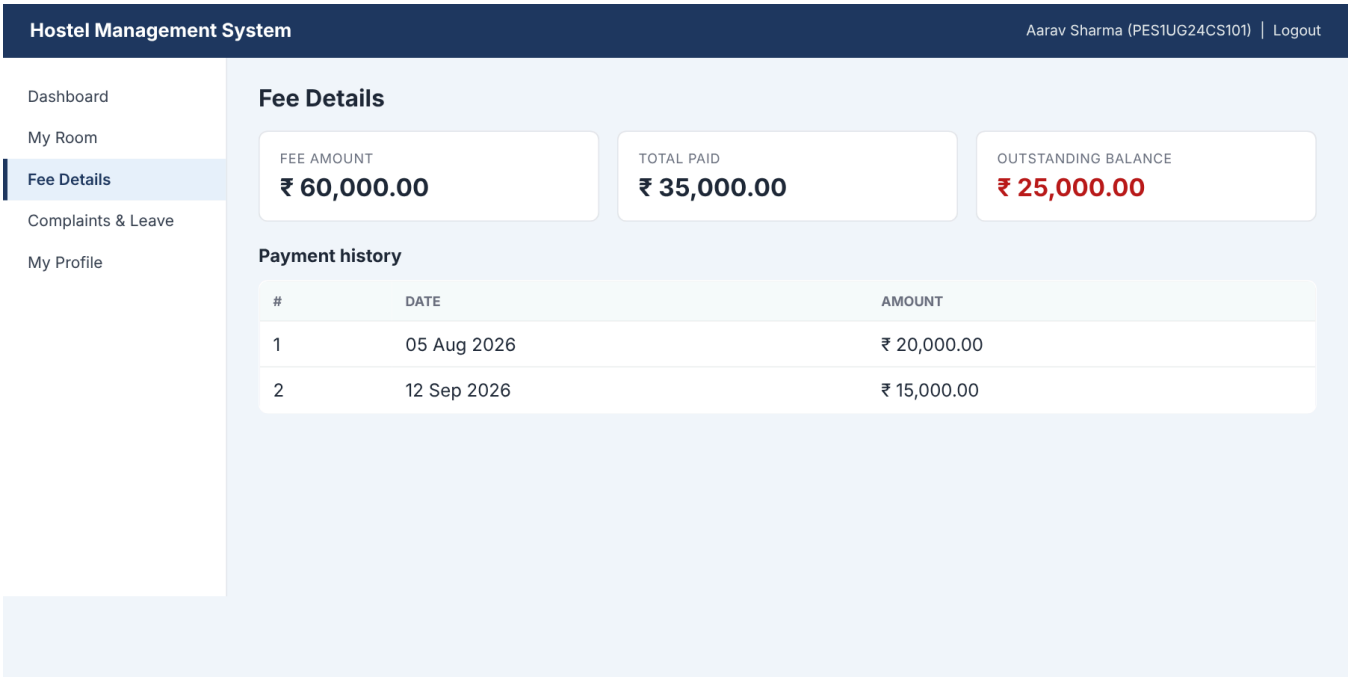


Figure 3.6 – Fee Details screen (Student).

3.1.7 Fees (Admin)

Allows the admin to record a payment entry for a student and to view outstanding dues across all students. A payment that exceeds the outstanding balance is rejected with an error.

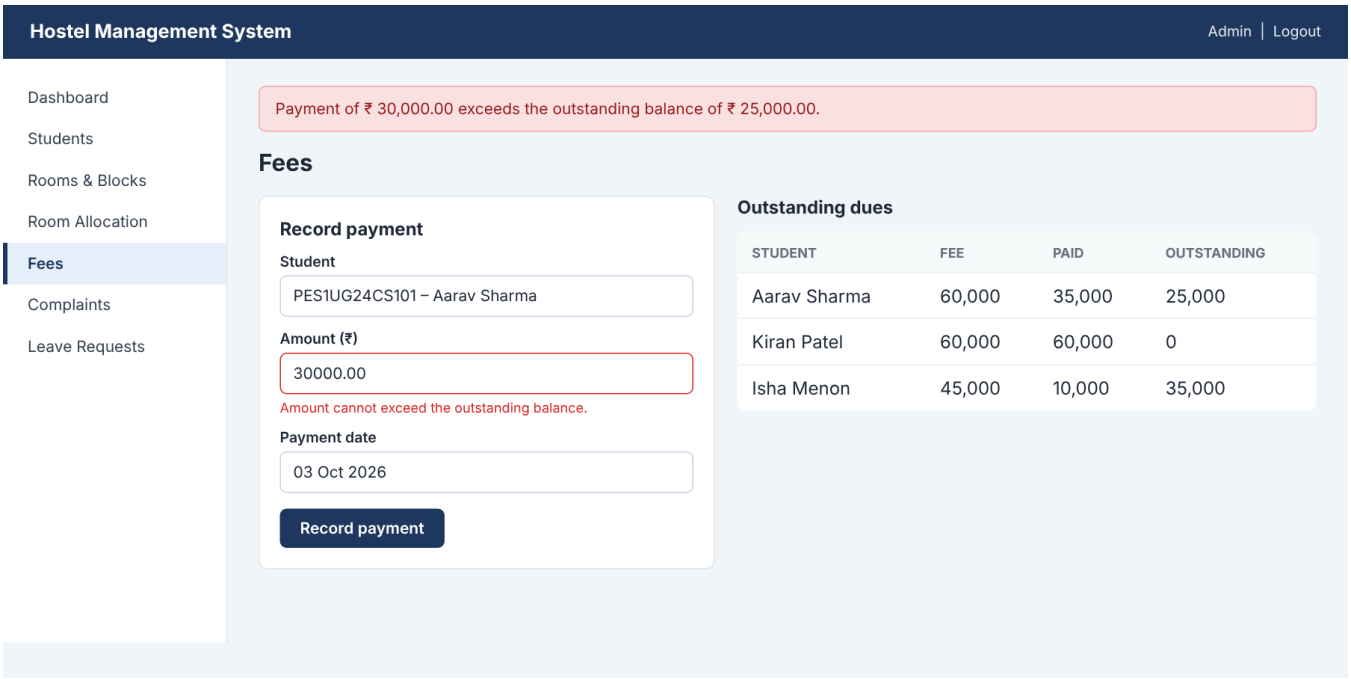


Figure 3.7 – Fees screen (Admin).

3.1.8 Complaints & Leave (Student)

Allows the student to submit a maintenance complaint for their allocated room and a leave request with a date range and reason, and to track the status of each.

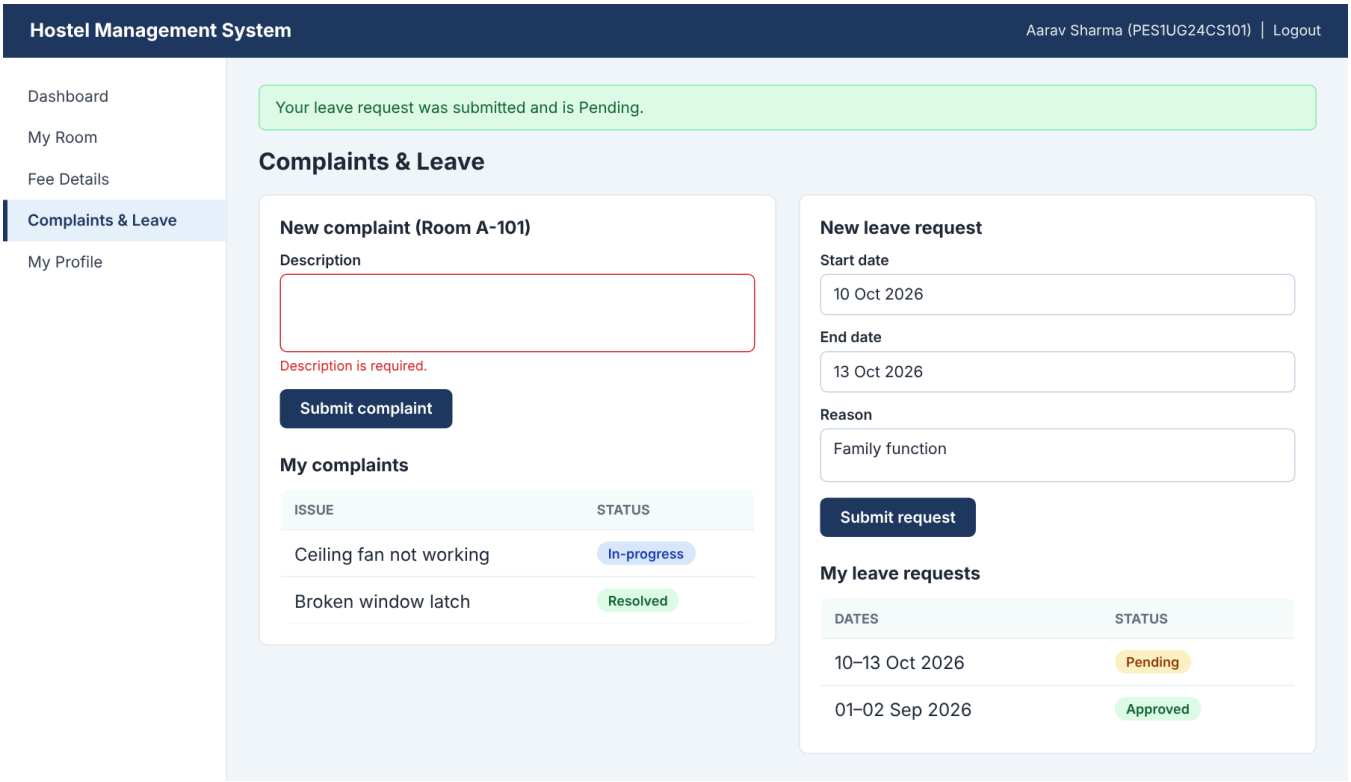


Figure 3.8 – Complaints & Leave screen (Student).

3.1.9 Complaints & Leave Requests (Admin)

Allows the admin to update the status of complaints and to approve or reject pending leave requests.

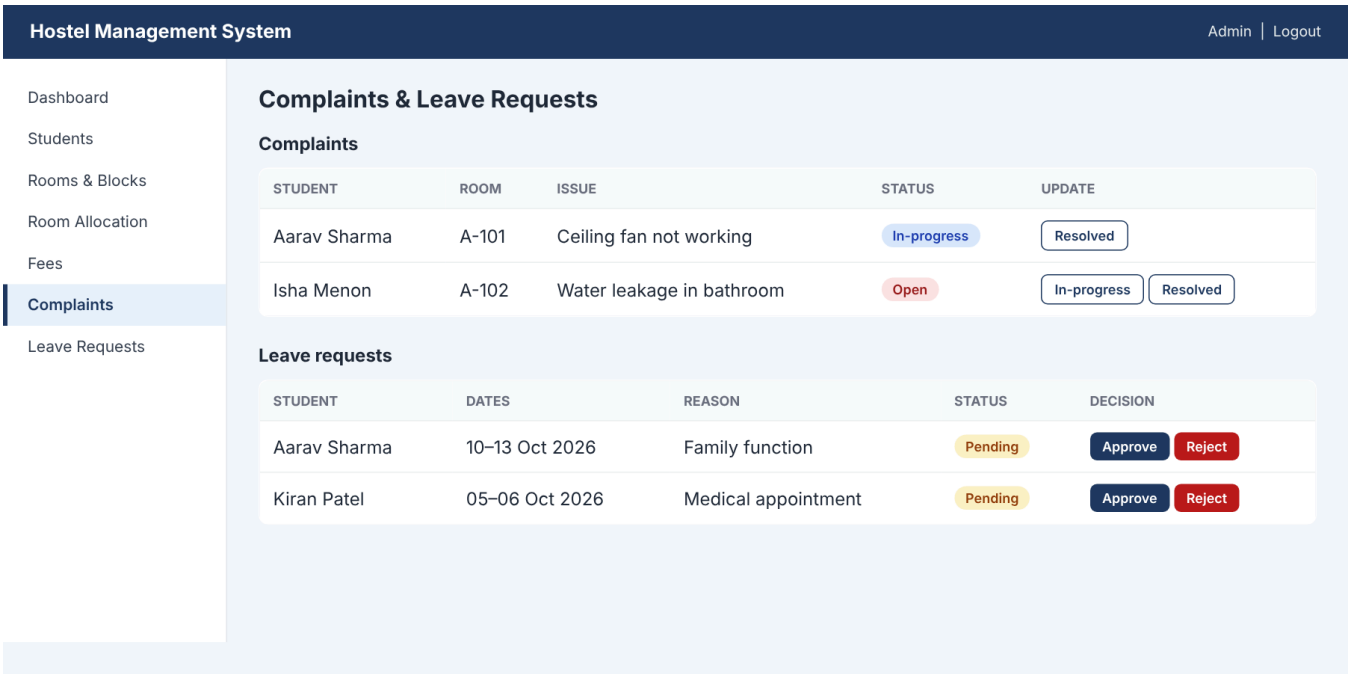


Figure 3.9 – Complaints & Leave Requests screen (Admin).

3.1.10 Screens added during implementation

The following screens follow the same layout and conventions as above and were added while implementing the requirements; no separate prototype figure was drawn for them:

- **My Profile (Student):** shows SRN, email, account status and registration date, and lets the student edit their name and guardian contact (REQ-5).
- **Dues Report (Admin):** lists every student with a fee record, sorted by outstanding balance, with totals and a "Show only students with dues" filter (REQ-16).
- **Audit Log (Admin):** lists audit entries (user, action, details, IP address, time) with a filter by action, 50 entries per page (SEC-6).

3.2 Software Interfaces

- **Backend framework:** Django 5.2 (LTS), running on Python 3.x.
- **Database engine:** PostgreSQL 16, chosen for its relational integrity between the Student, Room, Fee, and related entities. A local SQLite file is used only for development runs without Docker.
- **ORM:** Django ORM is used for all database access; no raw SQL is required for normal operations.
- **Authentication:** Django's built-in authentication and session framework.
- No external payment gateway library or SDK is used in this version of the system.

3.3 Communications Interfaces

The system uses a server-rendered architecture: Django views render HTML templates and the browser communicates with the server over HTTP (HTTPS in deployment) using standard form submissions. No separate REST API or separate frontend application is used in this version.

Email/SMS notification interface: Not applicable.

3.4 Hardware Interfaces

The system requires no special hardware. Users access it through a standard computer or mobile device with a modern web browser, and a keyboard/mouse or touch input.

4. Analysis Models

4.1 Use Case Diagrams

The use case diagrams below show the two actors, Student and Admin, and the twelve use cases (UC-01 to UC-12) listed in Appendix D. Both actors use UC-01 (Register / Login), since login applies to both roles; registration applies to Students only.

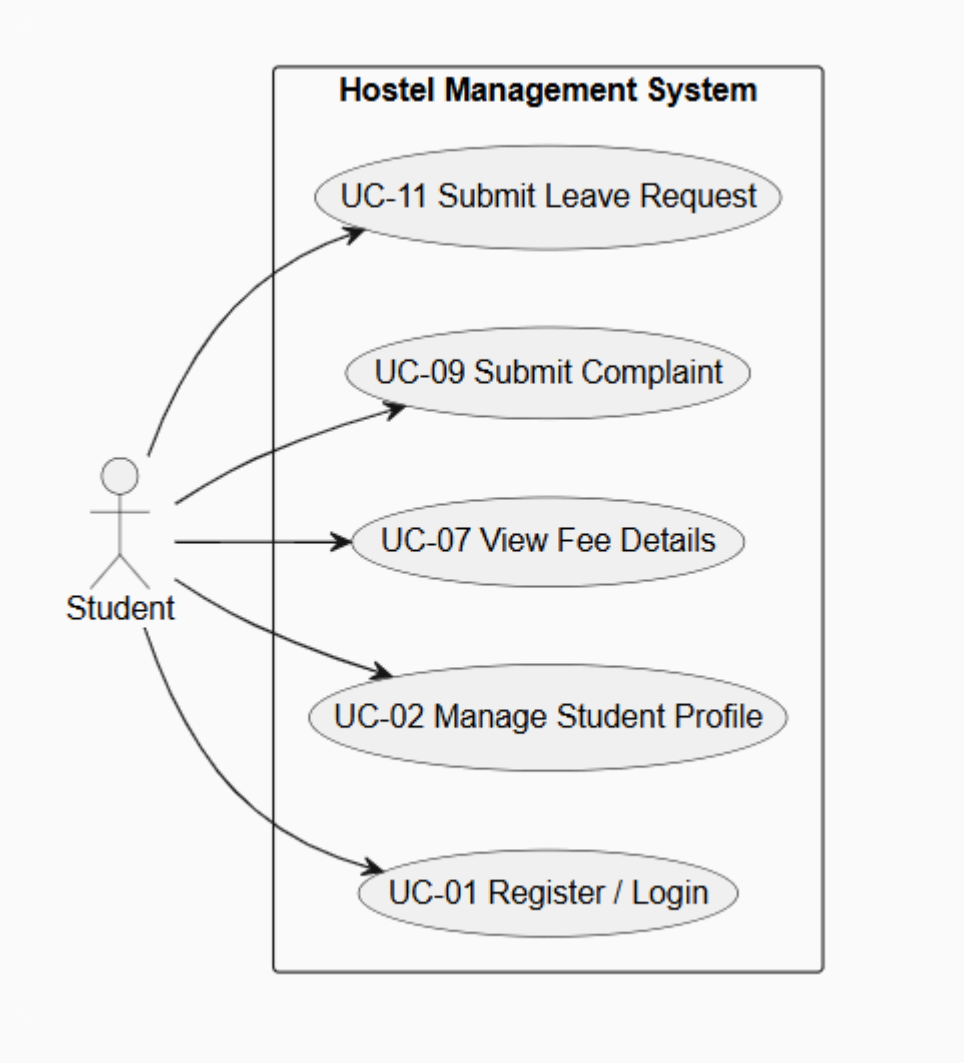


Figure 4.1a – Use cases of the Student actor.

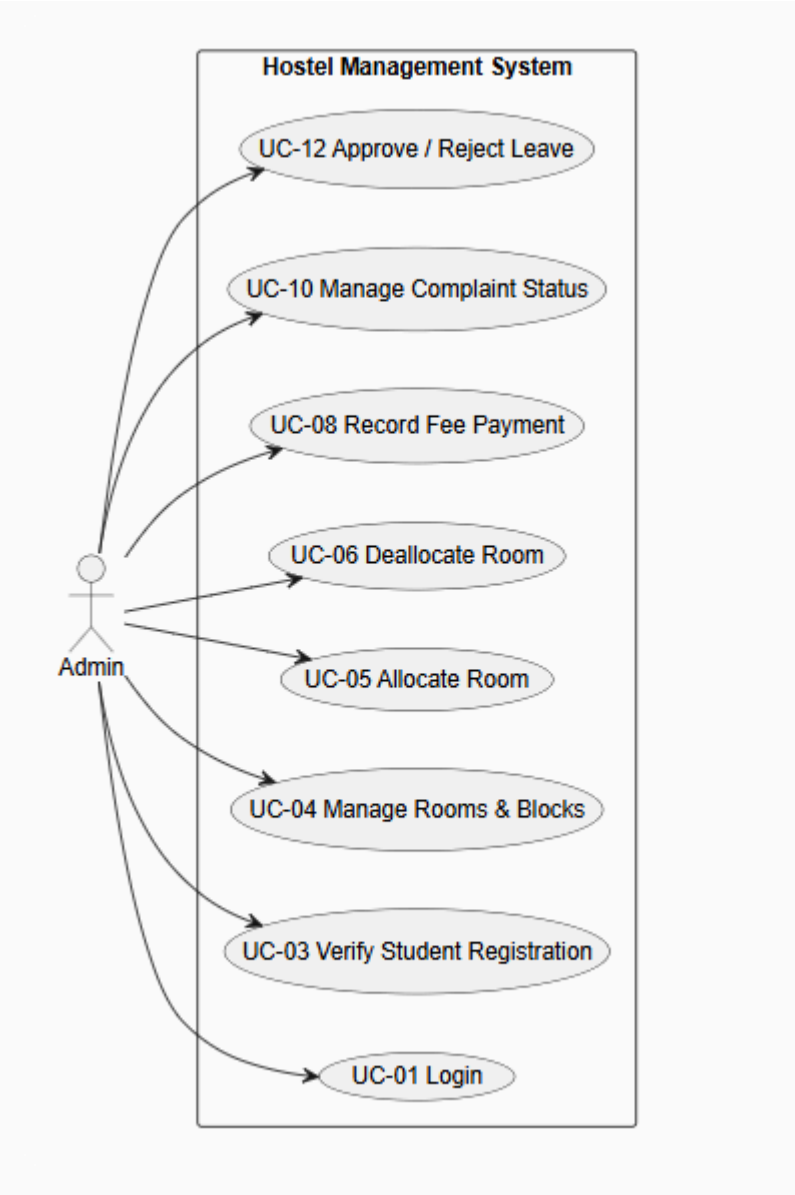


Figure 4.1b – Use cases of the Admin actor.

4.2 ER Diagram

The ER diagram below shows the core entities: Student, Room, Block, Allocation, Fee, PaymentRecord, Complaint, LeaveRequest, and AuditEntry. A Block contains many Rooms. A Room may be allocated to many Students, up to its capacity (REQ-7), and a Student has at most one allocated Room. The allocation is stored as a separate Allocation record that is one-to-one with the Student and many-to-one with the Room (see Appendix E). Each Student has at most one Fee record, which may be set per student (REQ-11), and many PaymentRecords (REQ-12). The outstanding balance is calculated as the fee amount minus the sum of payment amounts (REQ-13) and is not stored. A Student files Complaints, each tied to a Room (REQ-17), and submits LeaveRequests (REQ-19). Admin users are stored in the same user table as Students, distinguished by the `role` attribute, and are therefore not a separate entity. AuditEntry records security-relevant events and refers to the user who performed them (SEC-6).

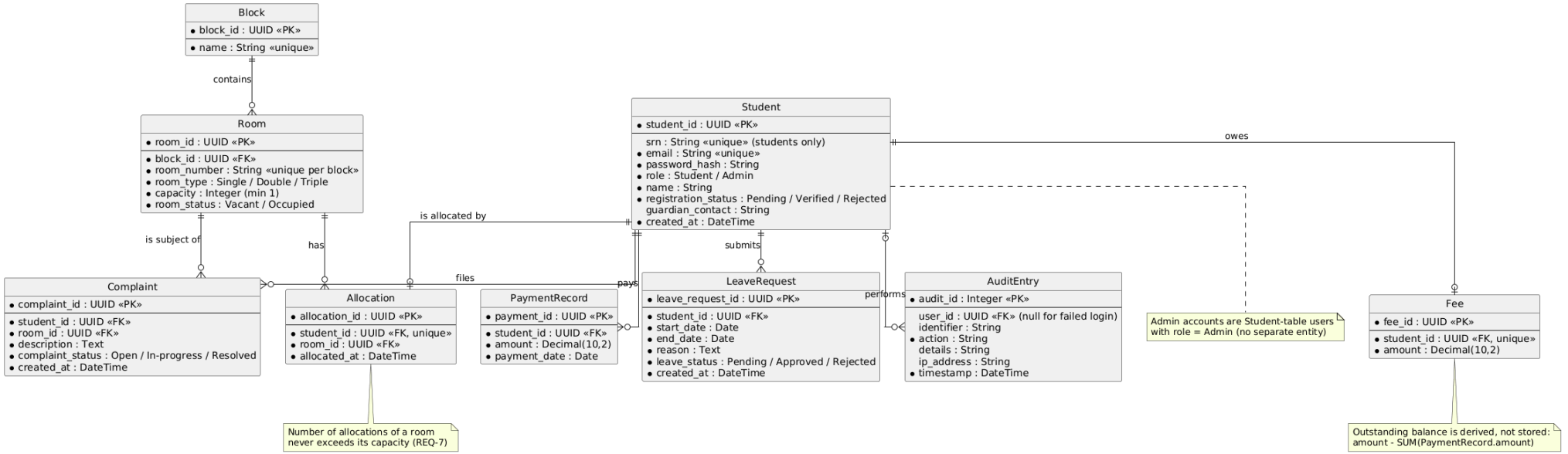


Figure 4.2 – Entity-relationship diagram of the Hostel Management System.

5. System Features

5.1 Student & Access Management

Description and Priority

Student registration (with admin verification), login for both roles, role-based access control, student profile management. **Priority: High** — every other feature depends on this.

Stimulus/Response Sequences

- "*Student submits registration form → account created in 'Pending' state → Admin verifies → account activated → student can log in.*"
- Invalid input: "*User submits invalid/duplicate email → validation error shown → form redisplayed.*"

Functional Requirements

ID	Requirement	Priority	Verification
REQ-1	The system shall allow a student to register using a unique email and SRN	High	Functional test
REQ-2	The system shall keep newly registered accounts in a "Pending" state until admin-verified.	High	Functional test
REQ-3	The system shall allow an admin to approve or reject a pending registration.	High	Functional test
REQ-4	The system shall enforce role-based access so students cannot access admin-only views.	High	Authorization test
REQ-5	The system shall allow a student to view and edit their own profile (contact, guardian info).	Medium	Functional test
REQ-21	The system shall allow a Verified user (Student or Admin) to log in with email (or SRN for students) and password, and shall reject invalid credentials and accounts that are not Verified with an error message.	High	Functional test

5.2 Room & Allocation Management

Description and Priority

Room/block setup, capacity and availability tracking, direct allocation and deallocation by the admin. **Priority: High.** *(No room-change-request workflow in this version — if a student needs a different room, the admin deallocates and reallocates directly.)*

Stimulus/Response Sequences

- **Allocation:** The Admin selects a verified student and an available room and submits the allocation → the system checks that the room has not reached its capacity → the student is assigned to the room → the room occupancy is updated and its status becomes "Occupied" when full → the student sees the room on their dashboard.
- **Allocation to a full room:** The Admin attempts to allocate a student to a room that is already at capacity → the system rejects the request → an error message is shown and no allocation is made.

- **Deallocation:** The Admin deallocates a student from a room (for example, on checkout) → the system removes the allocation → the room occupancy is reduced and its status becomes "Vacant" → the student's dashboard no longer shows a room.

Functional Requirements

ID	Requirement	Priority	Verification
REQ-6	The system shall allow an admin to define blocks and rooms with a maximum capacity.	High	Functional test
REQ-7	The system shall prevent allocation of a room beyond its defined capacity.	High	Negative test
REQ-8	The system shall allow an admin to allocate an available room to a verified student.	High	Functional test
REQ-9	The system shall allow an admin to deallocate a student from their room.	High	Functional test
REQ-10	The system shall update room occupancy status automatically on allocation/deallocation.	High	Integration test

5.3 Fee Management

Description and Priority

Fee structure setup and payment record tracking, expressed purely as amounts (no gateway, no receipts). **Priority: High.**

Stimulus/Response Sequences

- "*Admin sets a fee amount for a student → student sees the amount due → admin records a payment entry → system recalculates and displays the outstanding balance.*"

Functional Requirements

ID	Requirement	Priority	Verification
REQ-11	The system shall allow an admin to define a fee amount for a student.	High	Functional test
REQ-12	The system shall allow an admin to record a fee payment entry (amount, date) against a student.	High	Functional test
REQ-13	The system shall calculate the outstanding fee amount as fee due minus total recorded payments.	High	Unit test
REQ-14	The system shall prevent a recorded payment amount from exceeding the outstanding balance.	High	Negative test
REQ-15	The system shall allow a student to view their fee amount, payment history, and outstanding balance.	Medium	Functional test
REQ-16	The system shall allow an admin to view a report of outstanding dues across all students.	Medium	Functional test

5.4 Complaints & Leave Management

Description and Priority

This module allows students to submit and track maintenance complaints related to their allocated room. It also allows students to submit leave or outing requests for a specified date range. The Admin can view submitted

complaints, update their status, and approve or reject student leave requests.

Priority: Medium — supporting operational features.

Stimulus/Response Sequences

Complaint Submission:

Student submits a maintenance complaint → system validates the complaint details → complaint is created with status `Open` → Admin can view the complaint → Admin updates the complaint status to `In-progress` or `Resolved` → student can view the updated status.

Leave Request Submission:

Student submits a leave/outing request with the required date range and reason → system validates the request → leave request is created with status `Pending` → Admin reviews the request → Admin approves or rejects the request → student can view the decision.

Functional Requirements

ID	Requirement	Priority	Verification
REQ-17	The system shall allow a student to file a maintenance complaint tied to their allocated room.	High	Functional test
REQ-18	The system shall allow an admin to update complaint status as `Open`, `In-progress`, or `Resolved`.	High	Functional test
REQ-19	The system shall allow a student to submit a leave/outing request with a valid date range and reason.	High	Functional test
REQ-20	The system shall allow an admin to approve or reject a pending leave request.	High	Functional test

6. Other Nonfunctional Requirements

6.1 Performance Requirements

NFR-1: The system shall respond to standard page requests (dashboard, room list, fee view) within 2 seconds for 95% of requests with up to 10 concurrent users, measured with a load-testing tool (JMeter or Locust).

6.2 Safety Requirements

- Not applicable — the system does not control any safety-critical hardware or process, and no physical harm can result from a failure of the system.

6.3 Security Requirements

6.3.1 Security Objectives

- **SO-1 — Confidentiality:** Student personal, fee, complaint and leave data shall be visible only to the owning student and Admin.
- **SO-2 — Integrity:** Room allocation, fee and payment records shall be created or changed only by authorized Admin through validated input.

- **SO-3 — Accountability:** Privileged actions and failed logins shall be traceable to the user and time.

6.3.2 Security Requirements

ID	Security Requirement	Validation Approach
SEC-1	Passwords shall not be stored in plaintext.	Code/config review
SEC-2	Role-based access shall prevent students from accessing admin-only endpoints.	Authorization tests
SEC-3	The system shall validate and sanitize all user-supplied input before processing.	Negative/security tests
SEC-4	The system shall invalidate a user's session after logout or expiry, and reject any request made with an invalid or expired session.	Security test
SEC-5	The system shall protect all state-changing form submissions against cross-site request forgery using per-session tokens.	Security test (a POST without a token is refused)
SEC-6	The system shall record an audit entry (user, action, time) for failed logins, denied access, room allocation and deallocation, payment recording, and complaint and leave decisions, and shall not log passwords or session tokens.	Log review and system test

6.4 Software Quality Attributes

ID	Category	Requirement	Verification
NFR-2	Usability	A student shall be able to file a complaint in no more than 4 screens, starting from the dashboard.	Usability walkthrough
NFR-3	Reliability	An invalid or failed submission (allocation, payment, complaint, leave) shall not create or change any record, and multi-step updates (allocation with occupancy, payment with balance) shall be saved in a single transaction.	Negative and integration tests
NFR-4	Maintainability	Each functional module shall be a separate Django app with business rules in service functions, and unit tests shall reach at least 70% line coverage per module.	Code review and coverage report
NFR-5	Compatibility	The system shall work without functional differences on the latest versions of Chrome and Firefox.	Cross-browser walkthrough

6.5 Business Rules

- "*Only verified students can be allocated a room.*"
- "*A room cannot be allocated beyond its defined capacity.*"
- "*Only an admin can approve or reject a leave request.*"
- "*A recorded payment cannot exceed a student's outstanding fee balance.*"
- "*Only an admin can allocate or deallocate a room.*"
- "*Only a student with an allocated room can submit a maintenance complaint.*"
- "*A newly submitted complaint shall have the status 'Open'.*"
- "*Only an admin can update the status of a complaint.*"
- "*A newly submitted leave request shall have the status 'Pending'.*"
- "*A student can view only their own complaints and leave requests.*"

---

7. Other Requirements

7.1 Database Requirements

- The system shall use PostgreSQL as the database management system (Docker and CI). A local SQLite database is permitted for development runs only.
- The database shall maintain relationships between students, rooms, fees, payments, complaints, and leave requests.
- Required fields shall not accept null or empty values where the corresponding field is marked mandatory in Appendix B.
- The database shall maintain data consistency when complaints or leave requests are created or their statuses are updated.

7.2 Data Retention and Privacy

- Student information, complaint records, and leave request records shall be accessible only to authorized users according to their roles.
- The system shall retain complaint and leave request records for the duration required by the project or institutional requirements.
- The system shall not expose student information to unauthorized users.

7.3 Other Requirements

- No external payment gateway is required for this version of the system.
- No additional hardware or third-party service is required beyond the components specified in this SRS.

Appendix A: Glossary

Term	Definition
Student	A registered user who lives in or has applied to live in the hostel.
Admin	A privileged user who manages rooms, fees, complaints, and leave requests.
Room	A physical hostel room with a defined capacity, belonging to a block.
Block	A group of rooms (e.g. a building or wing) within the hostel.
Fee	The amount due from a student for their hostel stay.
PaymentRecord	An admin-entered record of an amount paid by a student toward their fee.
Complaint	A maintenance issue reported by a student for their allocated room and tracked by the system until it is resolved.
LeaveRequest	A request submitted by a student to obtain permission for leave or outing for a specified date range.
RBAC	Role-Based Access Control — restricting actions based on user role (Student/Admin).
SRN	Student Registration Number, a unique number assigned to each student by the university.
CSRF	Cross-Site Request Forgery, an attack that makes a logged-in user's browser send an unwanted request; per-session tokens help prevent it.
Audit log	A record of who performed a privileged action, what action was performed, and when.
NFR	Non-Functional Requirement, a measurable quality requirement such as performance, usability, or reliability.

Appendix B: Field Layouts

Student

Field	Type	Description	Mandatory
`student_id`	UUID/String	Unique identifier for the student	Yes
`name`	String	Full name of the student	Yes
`email`	String	Unique login email	Yes
`guardian_contact`	String	Parent/guardian contact number	Yes
`registration_status`	Enum/String	`Pending` or `Verified` or `Rejected`	Yes
`created_at`	DateTime	Date and time the account was registered	Yes
`srn`	String	Student's unique SRN	Yes
`role`	Enum/String	Student or Admin	Yes

`srn` and `guardian_contact` are mandatory only when `role` = Student. For Admin accounts they may be empty. The student's current room is not stored on the Student record; it is stored in the Allocation record below.

Allocation

Field	Type	Description	Mandatory
`allocation_id`	UUID/String	Unique identifier for the allocation	Yes
`student_id`	UUID/String	Allocated student (unique: a student has at most one allocation)	Yes
`room_id`	UUID/String	Room the student is allocated to	Yes
`allocated_at`	DateTime	Date and time of the allocation	Yes

Block

Field	Type	Description	Mandatory
`block_id`	UUID/String	Unique identifier for the block	Yes
`name`	String	Block/building name	Yes

Room

Field	Type	Description	Mandatory
`room_id`	UUID/String	Unique identifier for the room	Yes
`block_id`	UUID/String	Identifier of the block this room belongs to	Yes
`room_number`	String	Room number/label	Yes
`capacity`	Integer	Maximum number of students the room can hold (minimum 1)	Yes
`room_status`	Enum/String	Current status: `Vacant` or `Occupied`	Yes
`room_type`	String	Type of room (e.g. Single, Double)	Yes

Complaint

Field	Type	Description	Mandatory
`complaint_id`	UUID/String	Unique identifier for the complaint	Yes
`student_id`	UUID/String	Identifier of the student who submitted the complaint	Yes
`room_id`	UUID/String	Identifier of the room associated with the complaint	Yes
`description`	Text	Description of the maintenance issue	Yes
`complaint_status`	Enum/String	Current status: `Open`, `In-progress`, or `Resolved`	Yes
`created_at`	DateTime	Date and time when the complaint was created	Yes

LeaveRequest

Field	Type	Description	Mandatory
`leave_request_id`	UUID/String	Unique identifier for the leave request	Yes
`student_id`	UUID/String	Identifier of the student submitting the request	Yes
`start_date`	Date	Start date of the leave/outing period	Yes
`end_date`	Date	End date of the leave/outing period	Yes
`reason`	Text	Reason provided by the student for the leave request	Yes
`leave_status`	Enum/String	Current status: `Pending`, `Approved`, or `Rejected`	Yes
`created_at`	DateTime	Date and time when the leave request was created	Yes

Fee

Field	Type	Description	Mandatory
`fee_id`	UUID/String	Unique identifier for the fee record	Yes
`student_id`	UUID/String	Identifier of the student this fee applies to	Yes
`amount`	Decimal	Total fee amount due	Yes

PaymentRecord

Field	Type	Description	Mandatory
`payment_id`	UUID/String	Unique identifier for the payment entry	Yes
`student_id`	UUID/String	Identifier of the student the payment belongs to	Yes
`amount`	Decimal	Amount recorded in this payment entry	Yes
`payment_date`	Date	Date the payment was recorded	Yes

AuditEntry

Field	Type	Description	Mandatory
`audit_id`	Integer	Unique identifier for the audit entry	Yes
`user_id`	UUID/String	User who performed the action (empty for a failed login or after the user is deleted)	No

Field	Type	Description	Mandatory
`identifier`	String	Email typed on the login form, or email of the acting user	No
`action`	Enum/String	Login, failed login, logout, access denied, registration decision, room created, room allocated, room deallocated, fee set, payment recorded, complaint status changed, leave decision	Yes
`details`	String	Short description of the event (never contains passwords or session tokens)	No
`ip_address`	String	IP address of the request	No
`timestamp`	DateTime	Date and time of the event	Yes

Appendix C: Requirement Traceability Matrix

Status: **N** = not implemented, **P** = partially implemented or not yet verified, **A** = implemented and verified by the listed test cases. Test cases are defined in STP section 16; the automated tests run in CI on every pull request.

Req ID	Requirement short	Section ref / Design spec	Module	Test case(s)	Status (N/P/A)	Comments
REQ-1	Register with unique email and SRN	5.1 / SAD 4.3.1	Accounts	TC-ACC-03	A	Sprint 1
REQ-2	New accounts stay Pending	5.1 / SAD 4.3.1	Accounts	TC-ACC-04	A	Sprint 1
REQ-3	Admin approves or rejects registration	5.1 / SAD 4.3.1	Accounts	TC-ACC-04	A	Sprint 1
REQ-4	Role-based access	5.1 / SAD 3.9	Accounts	TC-SEC-01	A	Sprint 1
REQ-5	Student views and edits own profile	5.1 / SAD 4.3.1	Accounts	TC-ACC-05	A	Sprint 2
REQ-6	Define blocks and rooms with capacity	5.2 / SAD 4.3.2	Rooms	TC-ROOM-05	A	Sprint 1
REQ-7	Prevent allocation beyond capacity	5.2 / SAD 4.2.2	Rooms	TC-ROOM-02	A	Sprint 1
REQ-8	Allocate room to a Verified student	5.2 / SAD 4.2.2, 4.3.2	Rooms	TC-ROOM-01, TC-ROOM-03	A	Sprint 1
REQ-9	Deallocate a student from a room	5.2 / SAD 4.3.2	Rooms	TC-ROOM-04	A	Sprint 2
REQ-10	Automatic occupancy and status update	5.2 / SAD 4.2.2	Rooms	TC-ROOM-01, TC-ROOM-04	A	Sprint 1 and 2
REQ-11	Admin defines fee amount for a student	5.3 / SAD 4.3.3	Fees	TC-FEE-01	A	Sprint 1
REQ-12	Admin records a payment	5.3 / SAD 4.2.3, 4.3.3	Fees	TC-FEE-01	A	Sprint 1
REQ-13	Outstanding = fee minus payments	5.3 / SAD 4.2.3	Fees	TC-FEE-01	A	Sprint 1
REQ-14	Payment cannot exceed outstanding	5.3 / SAD 4.2.3	Fees	TC-FEE-02	A	Sprint 1
REQ-15	Student views own fee details	5.3 / SAD 4.3.3	Fees	TC-FEE-03	A	Sprint 2
REQ-16	Admin dues report	5.3 / SAD 4.3.3	Fees	TC-FEE-04	A	Sprint 2
REQ-17	Student files a complaint	5.4 / SAD 4.3.4	Complaints & Leave	TC-REQ-01	A	Sprint 1
REQ-18	Admin updates complaint status	5.4 / SAD 4.3.4	Complaints & Leave	TC-REQ-02	A	Sprint 2

Req ID	Requirement short	Section ref / Design spec	Module	Test case(s)	Status (N/P/A)	Comments
REQ-19	Student submits a leave request	5.4 / SAD 4.2.4	Complaints & Leave	TC-REQ-03	A	Sprint 1
REQ-20	Admin approves or rejects leave	5.4 / SAD 4.2.4	Complaints & Leave	TC-REQ-04	A	Sprint 2
REQ-21	Login	5.1 / SAD 4.2.1	Accounts	TC-ACC-01, TC-ACC-02, TC-ACC-04	A	Sprint 1
SEC-1	Passwords stored hashed	6.3 / SAD 3.9	Accounts	TC-SEC-06	A	Sprint 1
SEC-2	Role-based access for admin pages	6.3 / SAD 3.9	Accounts	TC-SEC-01	A	Authorization matrix covers every page (Sprint 2)
SEC-3	Input validation and sanitising	6.3 / SAD 3.9	All modules	TC-SEC-03	A	Sprint 2
SEC-4	Session expiry and rejection	6.3 / SAD 4.2.1	Security	TC-SEC-02	A	30-minute idle timeout (Sprint 2)
SEC-5	CSRF protection	6.3 / SAD 3.9	Web UI	TC-SEC-05	A	Sprint 1
SEC-6	Audit log	6.3 / SAD 3.9, 4.3.5	Audit	TC-SEC-04	A	Sprint 2
NFR-1	Response time	6.1 / SAD 3.6	All modules	TC-PERF-01	A	Locust run, see docs/testing/performance_results.md
NFR-2	Complaint in 4 screens	6.4 / SAD 4.5	Web UI	TC-USE-01	A	See docs/testing/usability_nfr2.md
NFR-3	Atomic updates	6.4 / SAD 4.2.2	Rooms, Fees, Complaints & Leave	TC-REL-01	A	Sprint 2
NFR-4	Modular code, 70% coverage	6.4 / SAD 4.1	All modules	TC-COV-01	A	94% to 100% per module
NFR-5	Chrome and Firefox compatibility	6.4 / SAD 3.6	Web UI	TC-BRW-01	A	See docs/testing/browser_compatibility.md

Appendix D: Use Cases

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register / Login	Student	Application available	Authenticated session
UC-02	Manage Student Profile	Student	Logged in	Profile updated
UC-03	Verify Student Registration	Admin	Admin authenticated	Account activated/rejected
UC-04	Manage Rooms & Blocks	Admin	Admin authenticated	Room/block created or updated
UC-05	Allocate Room	Admin	Room available, student verified	Room assigned
UC-06	Deallocate Room	Admin	Student currently allocated	Room vacated
UC-07	View Fee Details	Student	Logged in	Fee amount + outstanding balance shown
UC-08	Record Fee Payment	Admin	Admin authenticated	Payment recorded, balance updated
UC-09	Submit Complaint	Student	Logged in, allocated a room	Complaint created
UC-10	Manage Complaint Status	Admin	Admin authenticated, complaint exists	Complaint status updated
UC-11	Submit Leave Request	Student	Logged in	Leave request created with `Pending` status
UC-12	Approve / Reject Leave	Admin	Admin authenticated, pending leave request exists	Leave request status updated

Appendix E: Changes After the Phase-1 Baseline

All requirements in this document were implemented without change to their meaning. The following points changed between the Phase-1 baseline (v1.3) and the final system. They are design and environment decisions; the full list with reasons is in SAD section 4.7.

#	Baseline (v1.3)	Final system	Reason
1	The student's room stored as `room_id` on the Student record	A separate Allocation record, one-to-one with the Student	Keeps the Accounts module independent of the Rooms module; same rule: a student has at most one room
2	PostgreSQL only	PostgreSQL 16 in Docker and CI; a local SQLite file for development runs without Docker	Lets every member run and test the project without installing PostgreSQL
3	CI with Jenkins	CI with GitHub Actions (tests on PostgreSQL, migration check, per-module coverage)	Runs on every pull request with no separate CI server to host
4	No screen for audit entries	Admin Audit Log page with a filter by action	Lets the admin review SEC-6 entries without database access
5	SEC-4 did not fix a timeout	Sessions expire after 30 minutes without a request, and when the browser is closed	Concrete value needed to implement and test SEC-4

#	Baseline (v1.3)	Final system	Reason
6	No rule on lowering a fee or on future payment dates	A fee cannot be set below the amount already paid, and a payment date cannot be in the future	Keeps the outstanding balance (REQ-13, REQ-14) valid in every case